Programski prevodioci

08 Generisanje koda

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

```c
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
   return 0;
}
```

test-ok1.mc

```
f:
   PUSH   %14
   MOV    %15,%14
   SUBS   %15,$4,%15
@f_body:
   MOV    $3,-4(%14)
   ADDS   8(%14),-4(%14),%0
   MOV    %0,%13
   JMP    @f_exit
@f_exit:
   MOV    %14,%15
   POP    %14
   RET
main:
   PUSH   %14
   MOV    %15, %14
   SUBS   %15,$4,%15
@main_body:
   PUSH   $7
   CALL   f
   ADDS   %15,$4,%15
   MOV    %13,%0
   MOV    %0,-4(%14)
   MOV    -4(%14), %13
   JMP    @main_exit
@main_exit:
   MOV    %14,%15
   POP    %14
   RET
```

make: output.asm □ <filename>.asm

test-ok1.asm

hipsim

program koji simulira izvršavanje na hipotetskoj masini

.asm fajlovi → hipsim
→ rezultat izvršavanja (brzo testiranje)
→ korak po korak izvršavanje (stanje memorije i registri)

make

```
Line#  Addr    [> - PC, * - breakpoint]
 24   [  18]                    MOV -4(%14), %13
 25   [  19]                    JMP @main_exit
 26   [  20] @main_exit:
 27   [  20]                    MOV %14, %15
 28   [  21]                    POP %14
 29   [  22]                    RET
 30   [  23]                    HALT
 31 > [  24]                    PUSH $0
 32   [  25]                    CALL main
 33   [  26]                    HALT

Global variables (name[address]=value)

Registers
PC=24     Flags[ZSCO]=[0000]
%0 =0     %1 =0     %2 =0     %3 =0
%4 =0     %5 =0     %6 =0     %7 =0
%8 =0     %9 =0     %10=0     %11=0
%12=0     %13=0     %14=0     %15=1024

Stack ([address]=value)
Press any key to continue, ctrl+c for exit...
```

```
Line#  Addr    [> - PC, * - breakpoint]
 24   [  18]                    MOV -4(%14), %13
 25   [  19]                    JMP @main_exit
 26   [  20] @main_exit:
 27   [  20]                    MOV %14, %15
 28   [  21]                    POP %14
 29   [  22]                    RET
 30   [  23]                    HALT
 31   [  24]                    PUSH $0
 32 > [  25]                    CALL main
 33   [  26]                    HALT

Global variables (name[address]=value)

Registers
PC=25     Flags[ZSCO]=[0000]
%0 =0     %1 =0     %2 =0     %3 =0
%4 =0     %5 =0     %6 =0     %7 =0
%8 =0     %9 =0     %10=0     %11=0
%12=0     %13=0     %14=0     %15=1020

Stack ([address]=value)
[1020]=0
Press any key to continue, ctrl+c for exit...
```

```
Line#  Addr    [> - PC, * - breakpoint]
  9   [   6]                    JMP @f_exit
 10   [   7] @f_exit:
 11   [   7]                    MOV %14, %15
 12   [   8]                    POP %14
 13   [   9]                    RET
 14   [  10] main:
 15 > [  10]                    PUSH %14
 16   [  11]                    MOV %15, %14
 17   [  12]                    SUBS %15, $4, %15
 18   [  13] @main_body:

Global variables (name[address]=value)

Registers
PC=10     Flags[ZSCO]=[0000]
%0 =0     %1 =0     %2 =0     %3 =0
%4 =0     %5 =0     %6 =0     %7 =0
%8 =0     %9 =0     %10=0     %11=0
%12=0     %13=0     %14=0     %15=1016

Stack ([address]=value)
[1020]=0
[1016]=26
Press any key to continue, ctrl+c for exit...
```

```
Line#  Addr    [> - PC, * - breakpoint]
 19   [  13]                    PUSH $7
 20   [  14]                    CALL f
 21   [  15]                    ADDS %15, $4, %15
 22   [  16]                    MOV %13, %0
 23   [  17]                    MOV %0, -4(%14)
 24 > [  18]                    MOV -4(%14), %13
 25   [  19]                    JMP @main_exit
 26   [  20] @main_exit:
 27   [  20]                    MOV %14, %15
 28   [  21]                    POP %14

Global variables (name[address]=value)

Registers
PC=18     Flags[ZSCO]=[0000]
%0 =10    %1 =0     %2 =0     %3 =0
%4 =0     %5 =0     %6 =0     %7 =0
%8 =0     %9 =0     %10=0     %11=0
%12=0     %13=10    %14=1012  %15=1008

Stack ([address]=value)
[1020]=0            <-FP+8
[1016]=26           <-FP+4
[1012]=0            <-FP
[1008]=10           <-FP-4
Press any key to continue, ctrl+c for exit...
```

```c
int f(int p) {
   int a;
   a = 3;
   return p + a;
}

int main() {
   int a;
   a = f(7);
   return 0;
}
```

miniC ⇔ C

```
f:
   PUSH   %14
   MOV    %15,%14
   SUBS   %15,$4,%15
@f_body:
   MOV    $3,-4(%14)
   ADDS   8(%14),-4(%14),%0
   MOV    %0,%13
   JMP    @f_exit
@f_exit:
   MOV    %14,%15
   POP    %14
   RET
main:
   PUSH   %14
   MOV    %15, %14
   SUBS   %15,$4,%15
@main_body:
   PUSH   $7
   CALL   f
   ADDS   %15,$4,%15
   MOV    %13,%0
   MOV    %0,-4(%14)
   MOV    -4(%14), %13
   JMP    @main_exit
@main_exit:
   MOV    %14,%15
   POP    %14
   RET
```

haj // micko

```
f:
   PUSH   %14
   MOV    %15,%14
   SUBS   %15,$4,%15
@f_body:
   MOV    $3,-4(%14)
   ADDS   8(%14),-4(%14),%0
   MOV    %0,%13
   JMP    @f_exit
@f_exit:
   MOV    %14,%15
   POP    %14
   RET
main:
   PUSH   %14
   MOV    %15, %14
   SUBS   %15,$4,%15
@main_body:
   PUSH   $7
   CALL   f
   ADDS   %15,$4,%15
   MOV    %13,%0
   MOV    %0,-4(%14)
   MOV    -4(%14), %13
   JMP    @main_exit
@main_exit:
   MOV    %14,%15
   POP    %14
   RET
```

haj // micko

```
f(int):
  push rbp
  mov rbp, rsp
  mov DWORD PTR [rbp-20], edi
  mov DWORD PTR [rbp-4], 3
  mov edx, DWORD PTR [rbp-20]
  mov eax, DWORD PTR [rbp-4]
  add eax, edx
  pop rbp
  ret
main:
  push rbp
  mov rbp, rsp
  sub rsp, 16
  mov edi, 7
  call f(int)
  mov DWORD PTR [rbp-4], eax
  mov eax, 0
  leave
  ret
```

x86-64 // gcc 14.2

```
f(int):
  push rbp
  mov rbp, rsp
  mov DWORD PTR [rbp-20], edi
  mov DWORD PTR [rbp-4], 3
  mov edx, DWORD PTR [rbp-20]
  mov eax, DWORD PTR [rbp-4]
  add eax, edx
  pop rbp
  ret
main:
  push rbp
  mov rbp, rsp
  sub rsp, 16
  mov edi, 7
  call f(int)
  mov DWORD PTR [rbp-4], eax
  mov eax, 0
  leave
  ret
```

x86-64 // gcc 14.2 // intel

```
f(int):
  pushq %rbp
  movq %rsp, %rbp
  movl %edi, -20(%rbp)
  movl $3, -4(%rbp)
  movl -20(%rbp), %edx
  movl -4(%rbp), %eax
  addl %edx, %eax
  popq %rbp
  ret
main:
  pushq %rbp
  movq %rsp, %rbp
  subq $16, %rsp
  movl $7, %edi
  call f(int)
  movl %eax, -4(%rbp)
  movl $0, %eax
  leave
  ret
```

x86-64 // gcc 14.2 // AT&T

GNU toolchain, Unix, GAS - GNU asembler

```
f(int):
  pushq %rbp
  movq %rsp, %rbp
  movl %edi, -20(%rbp)
  movl $3, -4(%rbp)
  movl -20(%rbp), %edx
  movl -4(%rbp), %eax
  addl %edx, %eax
  popq %rbp
  ret
main:
  pushq %rbp
  movq %rsp, %rbp
  subq $16, %rsp
  movl $7, %edi
  call f(int)
  movl %eax, -4(%rbp)
  movl $0, %eax
  leave
  ret
```

x86-64 // gcc 14.2

```
f(int):
  addiu   $sp,$sp,-24
  sw      $fp,20($sp)
  move    $fp,$sp
  sw      $4,24($fp)
  li      $2,3                        # 0x3
  sw      $2,8($fp)
  lw      $3,24($fp)
  lw      $2,8($fp)
  nop
  ...

main:
  addiu   $sp,$sp,-40
  sw      $31,36($sp)
  sw      $fp,32($sp)
  move    $fp,$sp
  li      $4,7                        # 0x7
  jal     f(int)
  nop
  ...
```

MIPS // gcc 14.2

reduced instruction set computing (RISC) arhitektura
specifične potrebe, nauka i nastava

```
f(int):
  pushq %rbp
  movq %rsp, %rbp
  movl %edi, -20(%rbp)
  movl $3, -4(%rbp)
  movl -20(%rbp), %edx
  movl -4(%rbp), %eax
  addl %edx, %eax
  popq %rbp
  ret
main:
  pushq %rbp
  movq %rsp, %rbp
  subq $16, %rsp
  movl $7, %edi
  call f(int)
  movl %eax, -4(%rbp)
  movl $0, %eax
  leave
  ret
```

x86-64 // gcc 14.2

```
f(int):
  pushq   %rbp
  movq    %rsp, %rbp
  movl    %edi, -4(%rbp)
  movl    $3, -8(%rbp)
  movl    -4(%rbp), %eax
  addl    -8(%rbp), %eax
  popq    %rbp
  retq

main:
  pushq   %rbp
  movq    %rsp, %rbp
  subq    $16, %rsp
  movl    $0, -4(%rbp)
  movl    $7, %edi
  callq   f(int)
  movl    %eax, -8(%rbp)
  xorl    %eax, %eax
  addq    $16, %rsp
  popq    %rbp
  retq
```

x86-64 // clang 19.1

```
f(int):
  pushq   %rbp
  movq    %rsp, %rbp
  movl    %edi, -4(%rbp)
  movl    $3, -8(%rbp)
  movl    -4(%rbp), %eax
  addl    -8(%rbp), %eax
  popq    %rbp
  retq

main:
  pushq   %rbp
  movq    %rsp, %rbp
  subq    $16, %rsp
  movl    $0, -4(%rbp)
  movl    $7, %edi
  callq   f(int)
  movl    %eax, -8(%rbp)
  xorl    %eax, %eax
  addq    $16, %rsp
  popq    %rbp
  retq
```

x86-64 // clang 19.1

```
TranslationUnitDecl
|-FunctionDecl <line:1:1, line:5:1> line:1:5 used f 'int (int)'
| |-ParmVarDecl <col:7, col:11> col:11 used p 'int'
| `-CompoundStmt <col:14, line:5:1>
|   |-DeclStmt <line:2:4, col:9>
|   | `-VarDecl <col:4, col:8> col:8 used a 'int'
|   |-BinaryOperator <line:3:4, col:8> 'int' lvalue '='
|   | |-DeclRefExpr <col:4> 'int' lvalue Var 0xbe44128 'a' 'int'
|   | `-IntegerLiteral <col:8> 'int' 3
|   `-ReturnStmt <line:4:4, col:15>
|     `-BinaryOperator <col:11, col:15> 'int' '+'
|       |-ImplicitCastExpr <col:11> 'int' <LValueToRValue>
|       | `-DeclRefExpr <col:11> 'int' lvalue ParmVar 0xbe43f40 'p' 'int'
|       `-ImplicitCastExpr <col:15> 'int' <LValueToRValue>
|         `-DeclRefExpr <col:15> 'int' lvalue Var 0xbe44128 'a' 'int'
`-FunctionDecl <line:7:1, line:11:1> line:7:5 main 'int ()'
  `-CompoundStmt <col:12, line:11:1>
    |-DeclStmt <line:8:4, col:9>
    | `-VarDecl <col:4, col:8> col:8 used a 'int'
    |-BinaryOperator <line:9:4, col:11> 'int' lvalue '='
    | |-DeclRefExpr <col:4> 'int' lvalue Var 0xbe44418 'a' 'int'
    | `-CallExpr <col:8, col:11> 'int'
    |   |-ImplicitCastExpr <col:8> 'int (*)(int)' <FunctionToPointerDecay>
    |   | `-DeclRefExpr <col:8> 'int (int)' lvalue Function 0xbe44018 'f' 'int (int)'
    |   `-IntegerLiteral <col:10> 'int' 7
    `-ReturnStmt <line:10:4, col:11>
      `-IntegerLiteral <col:11> 'int' 0
```

AST output

Compiler Explorer: godbolt.org

COMPILER EXPLORER Add... More Templates Sponsors JETBRAINS Google Share Policies Other

C++ source #1 C++

```cpp
int f(int p) {
    int a;
    a = 3;
    return p + a;
}

int main() {
    int a;
    a = f(7);
    return 0;
}
```

x86-64 gcc 14.2 (Editor #1) x86-64 gcc 14.2 Compiler options...

Output... Filter... Libraries Overrides Add new... Add tool...

```asm
f(int):
        pushq   %rbp
        movq    %rsp, %rbp
        movl    %edi, -20(%rbp)
        movl    $3, -4(%rbp)
        movl    -20(%rbp), %edx
        movl    -4(%rbp), %eax
        addl    %edx, %eax
        popq    %rbp
        ret
main:
        pushq   %rbp
        movq    %rsp, %rbp
        subq    $16, %rsp
        movl    $7, %edi
        call    f(int)
        movl    %eax, -4(%rbp)
        movl    $0, %eax
        leave
        ret
```

Output (0/0) x86-64 gcc 14.2 - 329ms (4081B) ~254 lines filtered Compiler License